INDEX

120 PAGES

Name : DEVISHREE.J Std. : III Sec. : CSE

Roll No. : 702 Sub : Comp.Networks

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	20/7/26	BASIC LINUX Commands		
2	28/7/26	Different types of Network Cables		
3	3/8/26	CISCO PACKET TRACER INSTALLATIONS		
4	10/8/26	Setting UP a Local Area Network (LAN)		
5		Experiments on Packet capture tool: Wireshark		
6		Error Correction at Data link layer		
7		Flow Control at Data link layer		
8		Using NMAP Scans (ARP, ICMP, TCP/UDP) on the Try HackMe		
9		SUBNETTING in Cisco PACKET TRACER simulator		
10		Internetworking with routers in CISCO PACKET TRACER Simulator		
11		Routing at Network Layer.		
12		End-End Communication at Transport Layer		
13		Implement your own ping program		
14		Code using RAW sockets to implement packet sniffing		

20/1

EXNO:1 BASIC LINUX COMMANDS

i) Aim: Study of Various Network Commands used in Linux and Windows.

① arp-a: It shows the IP address of computer along with IP address and MAC address of computer

Output:
- gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3

② hostname:
It displays name of computes

Output: fedora

③ ipconfig/all:
Display information about TCP/IP connection including routes, Gateway, DNS, DHCP

Output:

Host Name: HDC0422263
Physical Address: 00-50-56-C0-00-08
DHCP enabled: Yes
IPv4 Address: 192.168.110.1 (Refered)
Subnet Mask: 255.255.255.0

④ nbtstat-a: It helps solve problems with NetBIOS

Output:

-a (adapter status) Lists remote machine's name table given its name

-A (adapter status) Lists remote machine's name table given its IP address.

-c (cache) Lists NBT's cache of remote [machine] names and their IP addresses

-n (names) Lists local NetBIOS names.

-r (resolved) Lists names resolved by broadcast and via WINS

nbtstat -n :

VMware Network Adapter VMnet 1:

Node IpAddress: [192.168.129.1] ScopeId: []

NetBIOS Local Name Table

Name	Type	Status
HDC04229263	<00> UNIQUE	Registered
WORKGROUP	<00> GROUP	Registered
HDC04229263	<20> UNIQUE	Registered

⑤ netstat: (net
network
as well as
etc

Output:

Proto

TCP

TCP

TCP

⑥ nslookup(
displa
partic
dom

Output:

Addres

⑦ pathpi
and

Out

Os

4. tcpdump :- This command is designed for captusing and displaying packets.

* To install :- # dnf install -y tcpdump

Output :- tcpdump -D

i) enp0s3 [Up, Running, Connected]
ii) lo [Up, Running, Loopback]
iii) usbmon1 (Raw USB traffic, bus number1)
iv) bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]

5. ping :-

Ping is a tool that verifies IP-level connectivity to another TCP/IP computer by sending Internet Control Message Protocol (ICMP) Echo Request messages. It is the primary TCP/IP command used to troubleshoot connectivity, reachability & name resolution

Output :- ping google.com

64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=1 ttl=255 time=6.29 ms

64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=2 ttl=255 time=7.77 ms

64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=3 ttl=255 time=6.01 ms

64 bytes from pnmaaa-ba-in-f14.1e100.net (142.251.221.206): icmp_seq=4 ttl=255 time=6.71 ms

al>

ether 08:00:07:36:02:94 txqueuelen 1000 (Ethernet)

RX packets 20539 bytes 27649339 (26.3 MiB)

RX errors 0 dropped 0 overruns 0 frame 0

TX packets 7723 bytes 515970 (503.8 KiB)

3. mtr

MTR (Matt's traceroute) is a program with command-line interface that serves as a network diagnostic and troubleshooting tool.

Output:-

i)

Keys: Help Display mode Restart statistics
Order of fields quit

Host	Packets		Pings				
	Loss%	Snt	Last	Avg	Best	Wrst	StDev
1. localhost	0.0%	295	0.1	0.1	0.0	0.7	0.1

ii) mtr -g google.com

-n, --no-dns do not resolve host names
-j, --json output json
-p, --split Split output
-x, --xml output xml
-l, --raw output raw format
-h, --help display this help and

4. tcpdump:-

* To ins

Output:-

i) en

ii)

iii)

iv

5. ping:-

to and

Contro

It

troub

des

O

64 b

64

64

64

Important Linux Commands

1. ip :- It show address information, manipulate routing, plus display network various devices, interfaces and tunnels

Eg:- ip address show

Output:- 1: lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000

link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00

inet 127.0.0.1/8 scope host lo

valid_lft forever preferred_lft forever

2: enp2s0: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000

link/ether 08:00:27:cb:77:eb brd ff:ff:ff:ff:ff:ff

inet6 fe80::cb77:eb33:81b7:72fc/64 scope link noprefix route.

2. ifconfig → Command in many sysadmin's tool belt for configuring & troubleshooting networks.

enp2s0: flags=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 1500

inet 172.16.9.200 netmask 255.255.252.0

broadcast 172.16.11.255

⑧ ping: (Packet INternet Grouper) command is the best way to test connectivity between 2 nodes.

Output: i) ping 172.16.8.1 (ping ip address)

Pinging 172.16.8.1 with 32 bytes of data:
Reply from 172.16.8.1: bytes=32 time<1ms TTL=64
Reply from 172.16.8.1: bytes=32 time<1ms TTL=64

Ping statistics for 172.16.8.1:
Packets: Sent=4, Received=4, Lost=0
Approximate round trip times in milli-seconds:
Minimum=0ms, Maximum=0ms, Avg=0ms

⑨ route:

used to manipulate IP routing table, setsup static routes to specific host or networks via an interface.

Output:

Kernel IP routing table

Destination	Gateway	Genmask	Flags	Metric	Ref	Use	Iface
default	-gateway	0.0.0.0	UG	100	0	0	emp
10.0.2.0	0.0.0.0	255.255.255.0	U	100	0	0	emp

1. ip: It
routing
inter

Eg ip

Output:
655
ge

2: enp
1500

inet

2. if co

emp

⑤ netstat: (network statistics) : Used for monitoring network connections both incoming & outgoing as well as viewing routing tables, interface etc

Output:-

Active connections

Proto	Local Address	Foreign Address	State
TCP	172.16.10.187:7680	Stylo:56600	TIME_WAIT
TCP	172.16.10.187:7680	HDC04288271:53790	TIME_WAIT
TCP	172.16.10.187:7680	172.16.10.149:49874	ESTABLISHED

⑥ nslookup (name server lookup):- It is used to display DNS details, such as IP address of particular computer, the MX records for domain

Output: Default Server: Unknown

Address: 172.16.8.1

⑦ pathping:- It is a unique command to Window's and combination of Ping and Tracert commands.

Output:

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n] [-p period] [-4] [-q num-queries] [-w timeout] [-6] target name

STP	Category 7	10Gbps	Disadvantages • Expensive • Greater installation effort	Gigabit Ethernet, 10G Ethernet (100m).	
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	• High bandwidth • Immune to interference • Low loss bandwidth • Versatile • Disadvantages: → Limited distance → Cost → Size is bulky	Speed of signal is 500m Television network High speed internet Connections	
fibre optics cable	Singlemode Multimode	100 Gbps	Advantages: • High speed • High bandwidth • High Security • Long distance Disadvantages • Expensive • Requires skilled installers	• Maximum distance of fibre optics cable is around 100 meters	

b) Make

Tools

- Ether but
- ✱ A
- ✱ Tw
- ✱ C

26/7

EXPERIMENT - 2

i) Aim: Study of different types of Network cables

ii) Understand different types of network cable

Different type of cables used in networking are:-

i) Unshielded Twisted Pair (UTP) Cable
ii) Shielded Twisted Pair (STP) Cable
iii) Coaxial Cable
iv) Fibre Optic Cable

Cable type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application/ Use	Image
UTP	Category 3	10 bps	Advantages *Cheaper in cost *Easy to install as they have smaller overall diameter. Disadvantages •More prone to (EMI) & noise	10 Base-T Ethernet Fast Ethernet, Gigabit Ethernet Fast Ethernet Gigabit Ethernet	
	Category 5	Upto 100 Mbps			
	Category 5e	1 Gbps			
STP	Category 6, 6a	10Gbps	Advantages: •Shielded •Faster than UTP •Less susceptible to noise & interference	Gigabit Ethernet 10G Ethernet (55 m) Widely used in data centres.	

NAME	UUID	TYPE	DEVICE
Wired Connection	2ff1365f-c2d9-3cb3-ab4f-5d636d7fe109	ethernet	eth0
lo	34fabove-983e-4941-a0ab-bcde5c7ffa38	loopback	lo

2. nmcli connection add type ethernet ifname eth0 con-name "Wired Communication 1"

Connection 'Wired Connection 1' (a0c990-7513-4c63-9bee-40edd01c4adc) successfully added.

nmcli connection show

NAME	UUID	TYPE	DEVICE
Wired Connection 1	d7d7365f-c2d9-3cb3-a64f	ethernet	eth0
lo	34faboo0e-983e-4941-a0ab-bcdc57	loopbxt	loop

3. nmcli connection up "Wired Connection 1"

Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/Authentication)

Student Observation

1. Which command is used to find reachability of host?
   Ans: Ping
2. Which command gives details of hops taken by packet to reach its destination?
   Ans: mtr command
3. Command displays IP configuration of your machine?
   Ans: ip address show
4. Which command TCP Port status on machine?
   Ans: netstat
5. Write command for assign & delete IP config?
   Ans: ip address add 192.168.1.254/24 dev eno53
   ip address del 198.168.1.254/24

28)-7

1) Aim:

2) Und

enp0s3: <BROADCAST, MULTICAST> mtu 1500 qdis
fd-codel DOWN mode DEFAULT group
default qlen 100
link/ether 08:00:27:02:df:8e brd ff:ff:ff:
ff:ff:ff

ii) ip link set enp0s3 promise on ip link show
enp0s3: <BROADCAST, MULTICAST, PROMISC>
mtu 1500 adisc fd_code
State DOWN DEFAULT qlen 1000
link/ether 08:00:27:02:df:8e brd
ff:ff:ff:ff:ff:ff

vi) ip route add default via 192.168.1.254
dev enp0s3

vii) ip route get
10.10.1.14 via 10.0.2.2 dev enp0s3 src
10.0.8.15 uid 1000

Configuring an Ethernet Connection by using

If you connect a host to network Over Ethernet, you can manage connection's setting on command

i) nmcli connection show

Balance ip

i) ip address add 192.168.1.254/24 dev enp503

o/p: Command executed successfully

To verify:

3: enp503: <BROADCAST, MULTICAST, UP, LOWER_UP> inet 192.168.1.254/24 scope global enp503

ii) ip address del 192.168.1.254/24 dev enp503

Output: To verify ip address show enp503

3: enp503: <BROADCAST, MULTICAST, UP, LOWER_UP>

iii) ip link set eth0 up

Output:

To verify: ip link show eth0

2: eth0: <BROADCAST, MULTICAST, UP, LOWER_UP> MTU 1500 qdisc fq_codel state UP group default qlen 1000

link/ether 08:00:27:02:df:se bcd ff:ff:ff:ff:ff:ff

alt name enX0500270 2dfse.

iv) ip link set enp0s3 down

ip link show enp0s3

Practical-3

i) Aim: To study the Packet tracer tool Installation and User Interface Overview

ii) Introduction:-

* A simulator, as name suggests, simulates network devices & its environment.

* Packet Tracer is an exciting network design, simulation and modelling tool.

i) It allows you to model complex systems without need for dedicated equipment
ii) It helps you to practice your network configuration and troubleshooting skills via computer or an Android (or) iOS based mobile device.
iii) It is available for both the Linux and Windows desktop environments.

iii) Installing Packet Tracer:-

To download Packet Tracer, go to https://www.netacad.com & login with your Cisco Networking Academy credentials; then, click on the Packet Tracer graphic and download the package appropriate for your operating system

Windows:-

Installation in Windows is pretty simple & straightfoward; the setup comes

Ans: Cross Cable

3. Which cable is used to connect a router/switch to a PC?

Ans: Straight cable

4. Category of twisted pair cable used in the lab

Ans Cat6 UTP Cable

5. Write down your understanding, challenges faced & output received?

Ans * Understanding: Learned how to make and test straight and cross cables.

* Challenges: Arranging wires correctly and crimping the RJ-45 connector.

* Output: Cable was made successfully and worked properly.

9/10/25

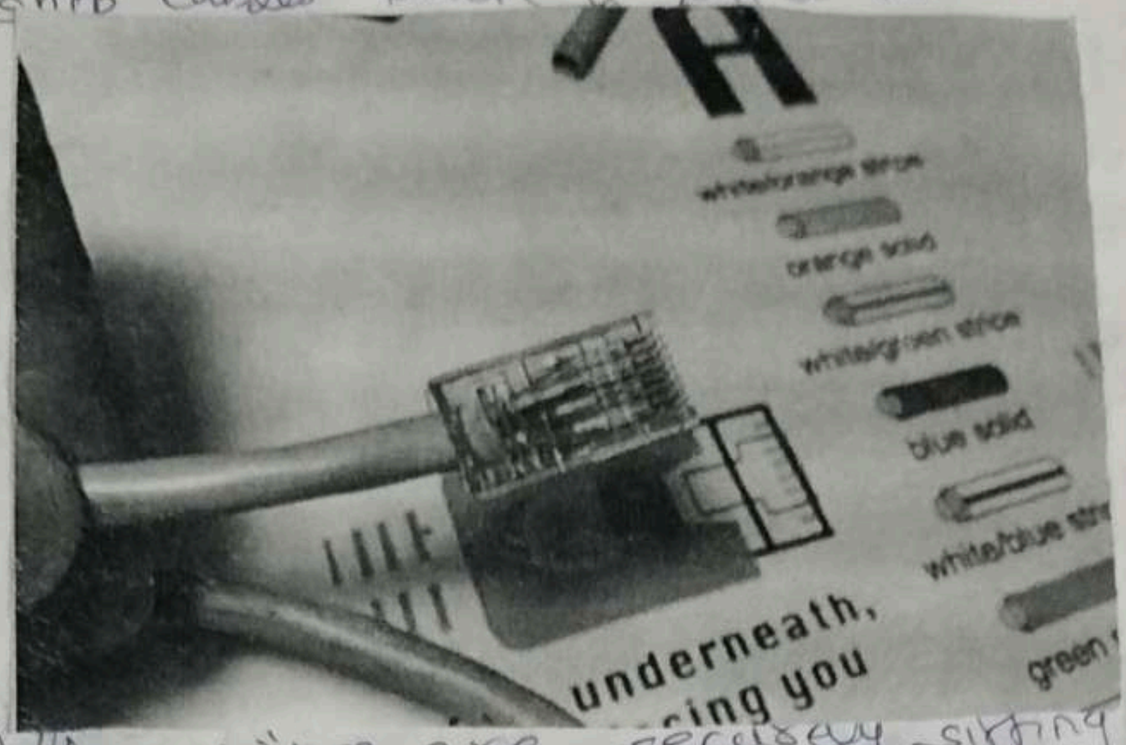
3/8

i) Aim:
and

ii) Introd
netw

iii)

Step5: Next, push cable right in. The notch at end of plug needs to be just over the cable shielding, & if it isn't that means you stripped off too much shielding. Simply grip cables back a little more.



Step6: After wires are securely sitting inside plug, insert it into crimping tool & push down. It should be shaped correctly, pushing too hard can crack fragile plastic plug.

Step7: Lastly, repeat for other end using diagB (to make crossover cables) / using diagA (to make straight through cable.

To test it, plug it in & attempt to connect 2 devices directly.

Student Observation:-

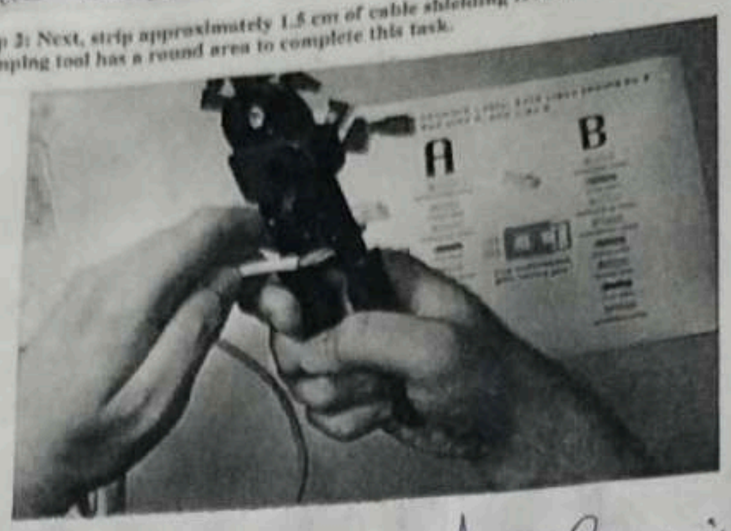
1. What is the difference between crosscable & straight cable?

Ans:- * Straight cable: Same wiring on both ends, used to connect different devices.

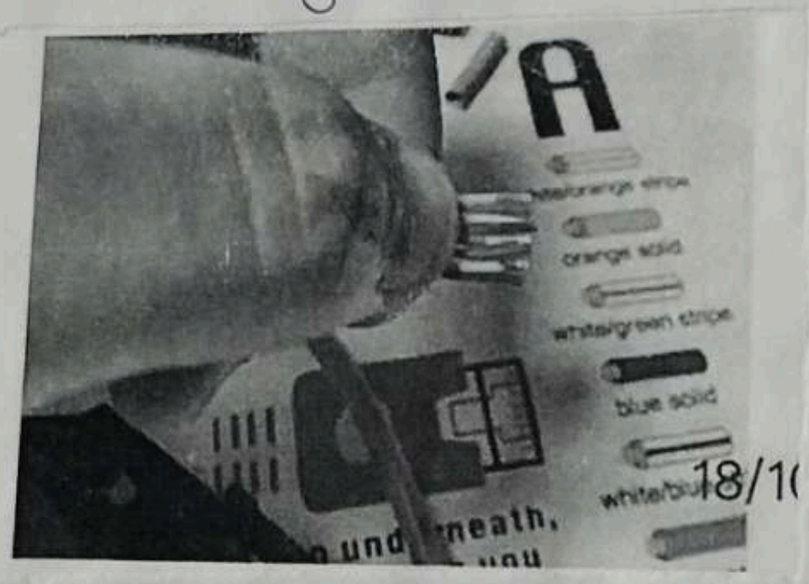
* Cross cable: Different wiring on both ends; used to connect similar devices.

2. Which cable is used to connect two PCs?

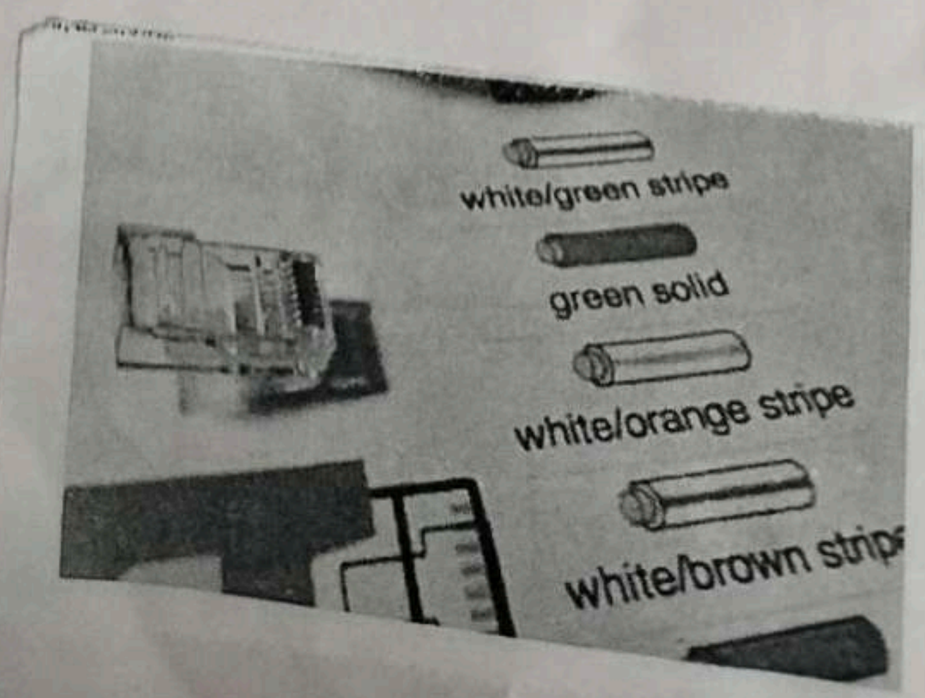
Step2: Next, strip approximately 1.5 cm of cable shielding from both ends. The coimping tool has a round area to complete this task.



Step3 After you will need to untangle wires, there should be four twisted pairs. Referencing back to the sheet, arrange them from top to bottom. One end should be in arrangement A & other in B



Step4: Once the order is correct, bunch them together in line & if there are any that stick out farther than others, snip them back to create an even level. To do so, hold plug with clip side facing away from you & have gold pins facing toward you.



Step
need
isn't
Simply

Step5:
ins
It s

Step7:
(to
st
To

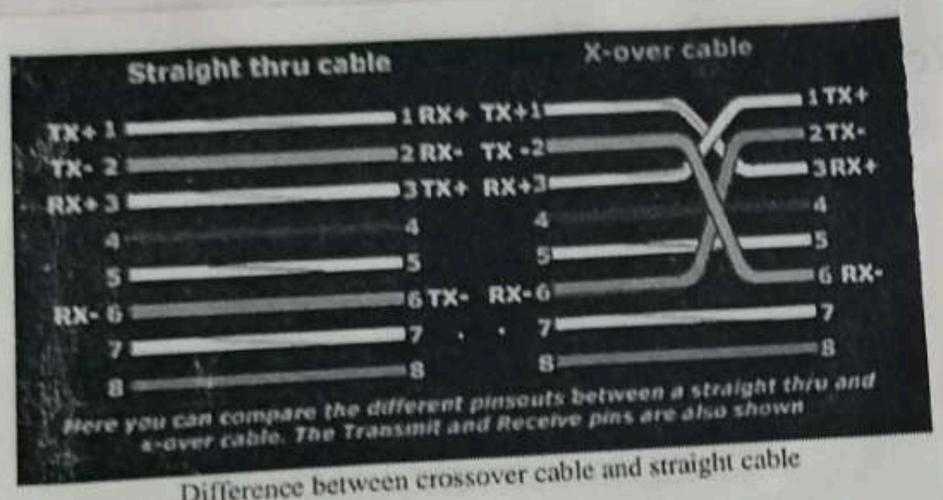
1.

2.

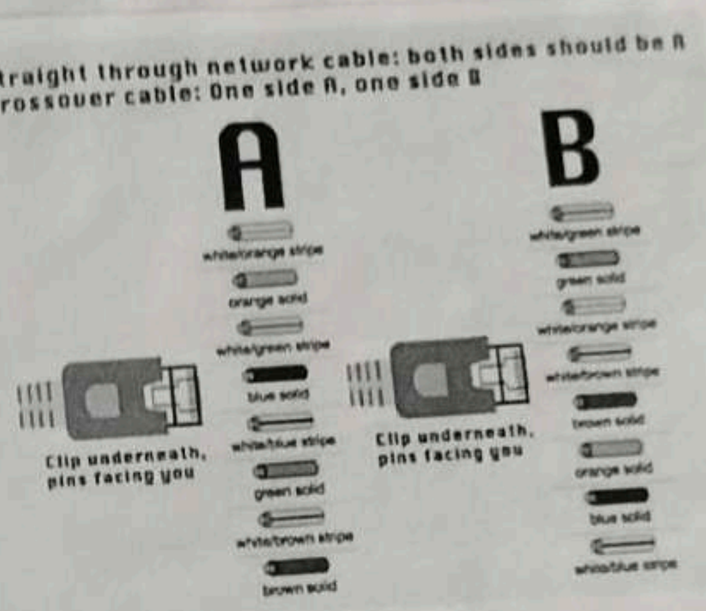
b) Make Your Own Ethernet Cross-Over Cable/Straight cable

Tools and parts needed:

- Ethernet cabling. CAT5e is certified for gigabit support, but Cat5 works as well.
* A crimping tool. All-in-one tool shaped to push down pins in the plugs strip and cut shielding off cables
* Two RJ45 plugs
* Optional two plug shields

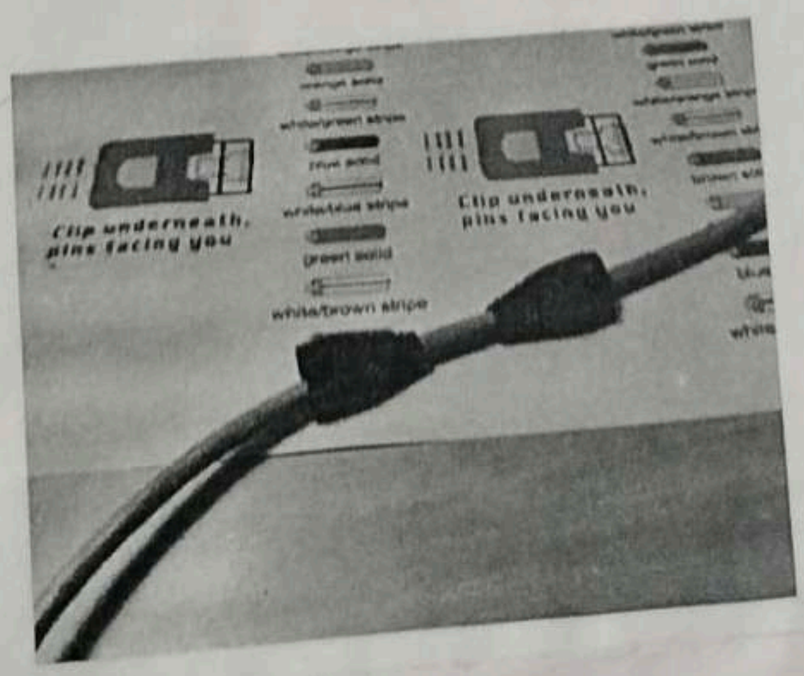


Difference between crossover cable and straight cable



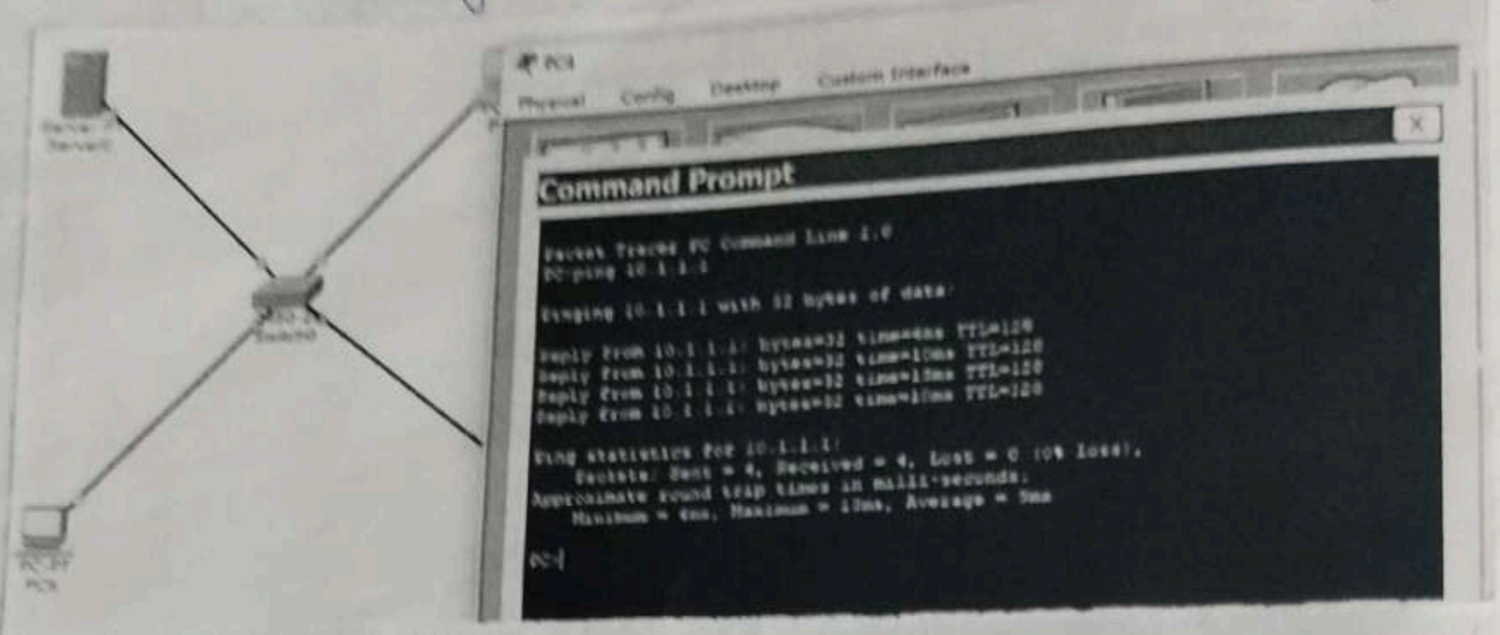
rint out the diagram below or have it handy as a reference

Step1: To start construction of device, begin by threading shields onto cable.



an IP address within same network range to be able to communicate with one another.

4. Network Diagram



5. IP Address Configuration Table

Device	IP address	Subnet Mask	Connected To
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch (Management)	10.1.1.5	255.0.0.0	VLAN 1 Interface

6. Implementation in Cisco Packet Tracer

Since physical hardware was limited, the LAN was also implemented & verified using CISCO Packet Tracer simulation following the steps

10/8

Ex No: 4 Setting Up a Local Area Network (LAN)

i) Aim: To design, setup & configure a Local Area Network (LAN) connecting four computers through network switch, assign static IP addresses to all devices, verify connectivity using ping command & demonstrate data sharing across network.

ii) Equipment Required:

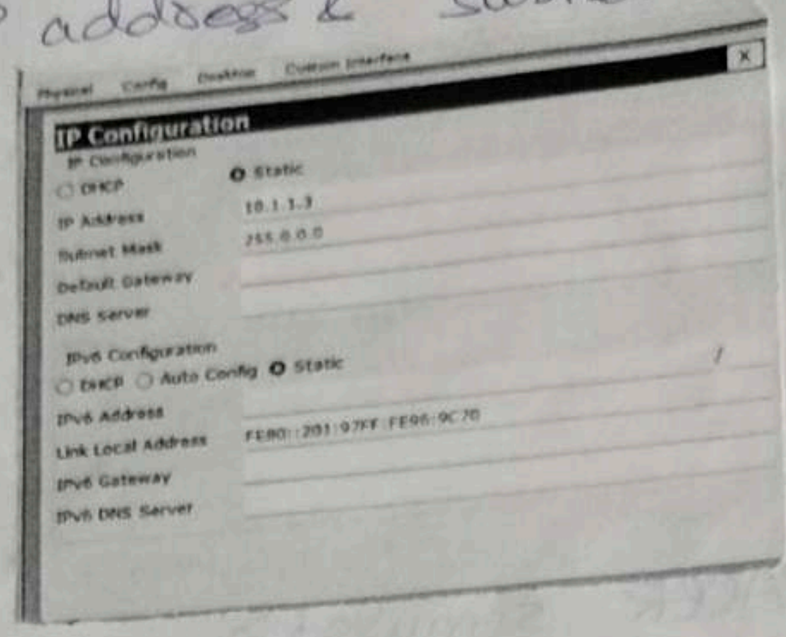
S.No	Equipment	Quantity	Specification
1	Personal Computers (PC)	4	Ethernet (NIC) enable
2	Network Switch	1	8/16/24 port, unmanaged/ managed
3	Ethernet Cables (Cat5e/Cat6)	4	straight through, RJ45 connectors

iii) Theory:

A Local Area Network (LAN) is a network that connects computers and devices within a limited geographical area such as home, office or laboratory allowing them to share resources such as files, printers & Internet connections.

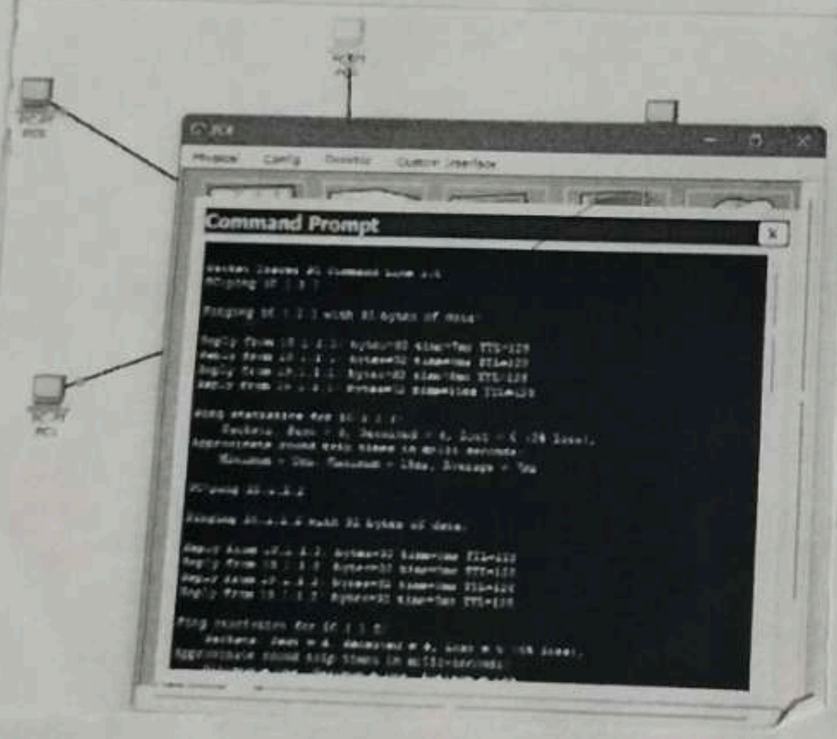
A star Topology is used in this experiment, where all end devices (PCs) are individually connected to central network switch. Every device on a LAN must be assigned

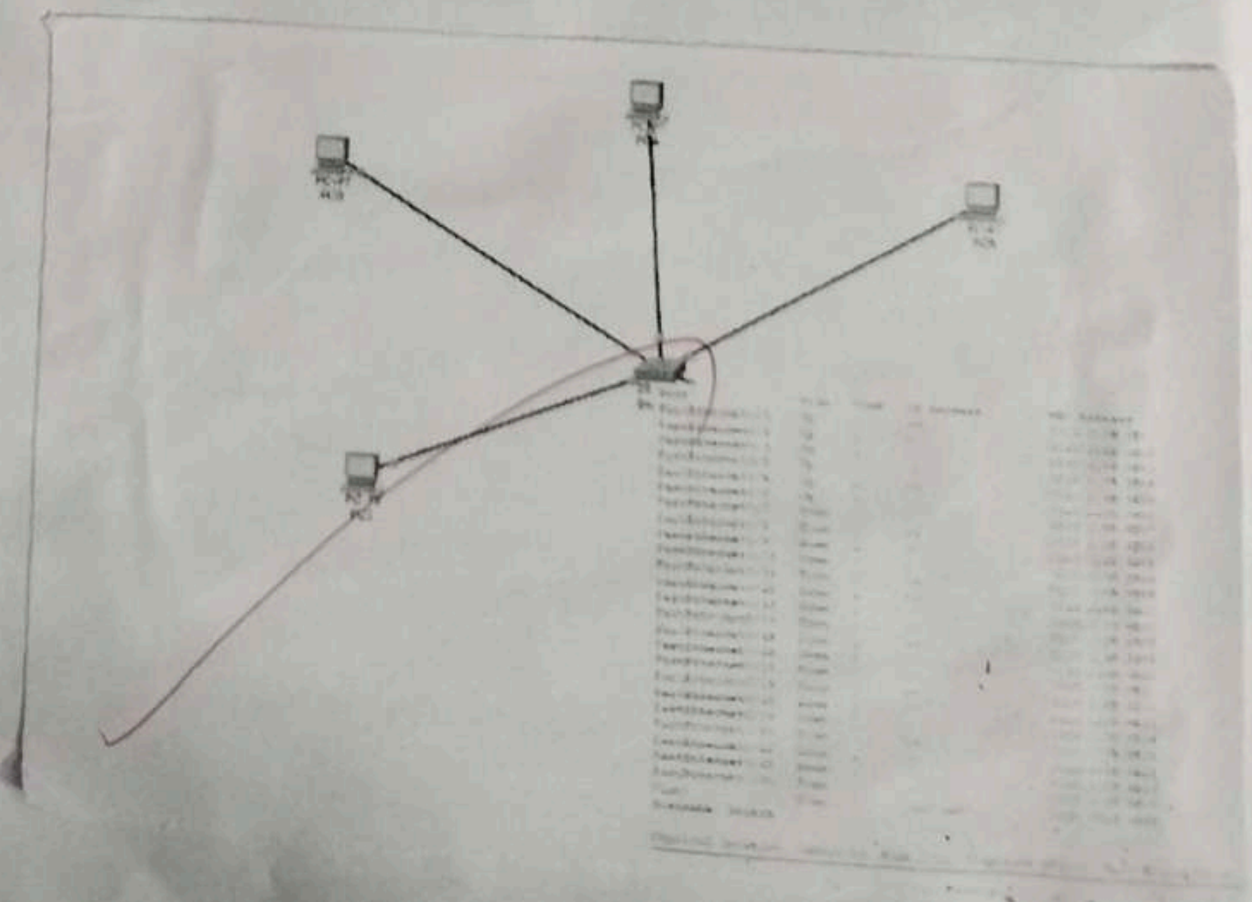
3. Click on PCs connected to hub, go to Desktop table, click on IP configuration, and enter an IP address & subnet mask.



4. Observe PDU from source PC to destination PC by selecting Realtime mode of simulation.

5. Observe how HUB & switch are forwarding the PDU & write your observation & conclusion about behaviours of Switch & HUB





4/8/24

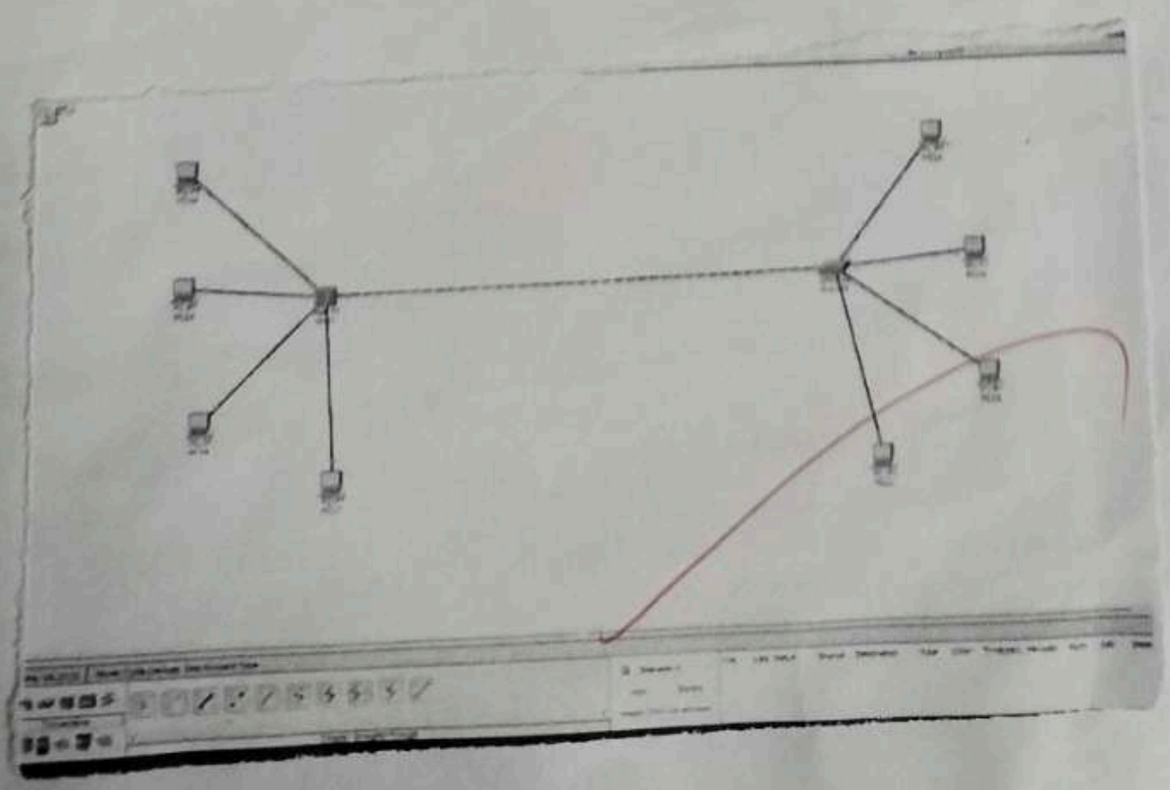
7. Network component box - It contains all of the network and end devices available with Packet Traces, which displays the different device models.

8. User-created packetbox - Users can highly create topology from this area, and results are displayed

8) Analyse behaviour of network devices using CISCO PACKET TRACER simulator.

1. From network component box, click drag & drop below components:
   a. 4 Generic PCs & one Hub
   b. Generic PCs & One switch

2. Click on Connections:
   a) Click on Copper Straight-Through cable
   b) Select one of PC & connect it to HUB.
The link LED should glow in green.
   c) Similarly connect all 4 PCs to switch using Copper straight - through cable.



in a file named Packettracer1

iv) User Interface Overview:
The layout of Packet Tracer is divided into several components. The components of the Packet Tracer interface are as follows:

1. Menu bar - This is a common menu found in all software applications; it is used to open, save, print, change preferences, and so on.

2. Main toolbar - This bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo and redo and on the right-hand side is an icon for entering network information for current network.

3. Logical / Physical workspace tabs - These tabs allow you to toggle between the Logical and Physical work areas.

4. Workspace - This area where topologies are created & simulations are displayed.

5. Common tools bar - This toolbar provides control for manipulating topologies.

6. Real-time / Simulation tabs - Used to toggle between real & simulation modes. Buttons also provided to control time.

15/9

Practical-5

i) Aim: Experiments on Packet Capture tool: Wireshark

ii) Capturing and Analysing Packets using wireshark tool

To filter, capture, view, packets in Wireshark Tool
Capture 100 packets from Ethernet: IEEE 802-3 LAN Interface and save it.

iii) Procedure:

* Select Local Area Connection in Wireshark
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture
* Save the packets.

① Create a Filter to display only TCP/ UDP packets, inspect the packets and provide flow graph.

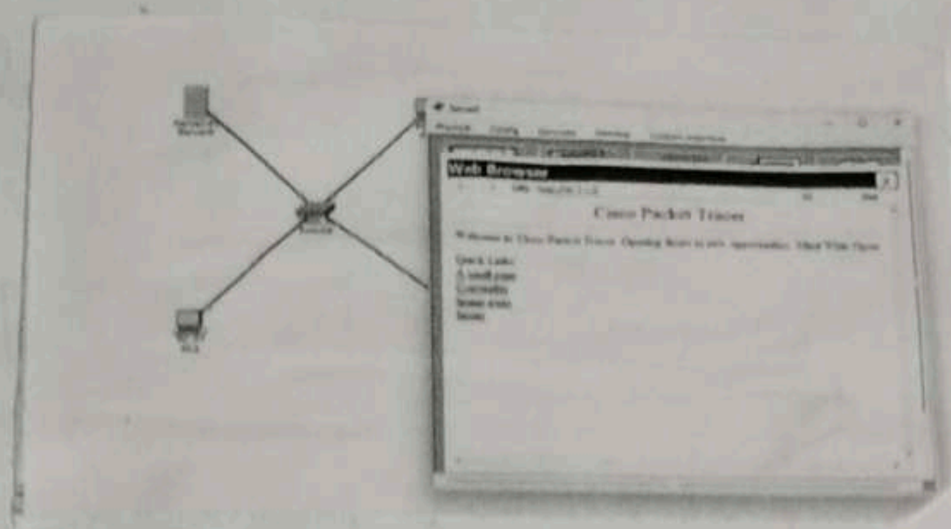
Procedure:

* Select Local Area Connection in Wireshark
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture
* Search TCP packets in search bar
* To see flow graph click

② Create a Filter to display only TCP/ UDP packets, inspect the packets and provide flow graph.

* Save

③ Cr
an



Step 1: Observe Packet Flow in Simulation Mode

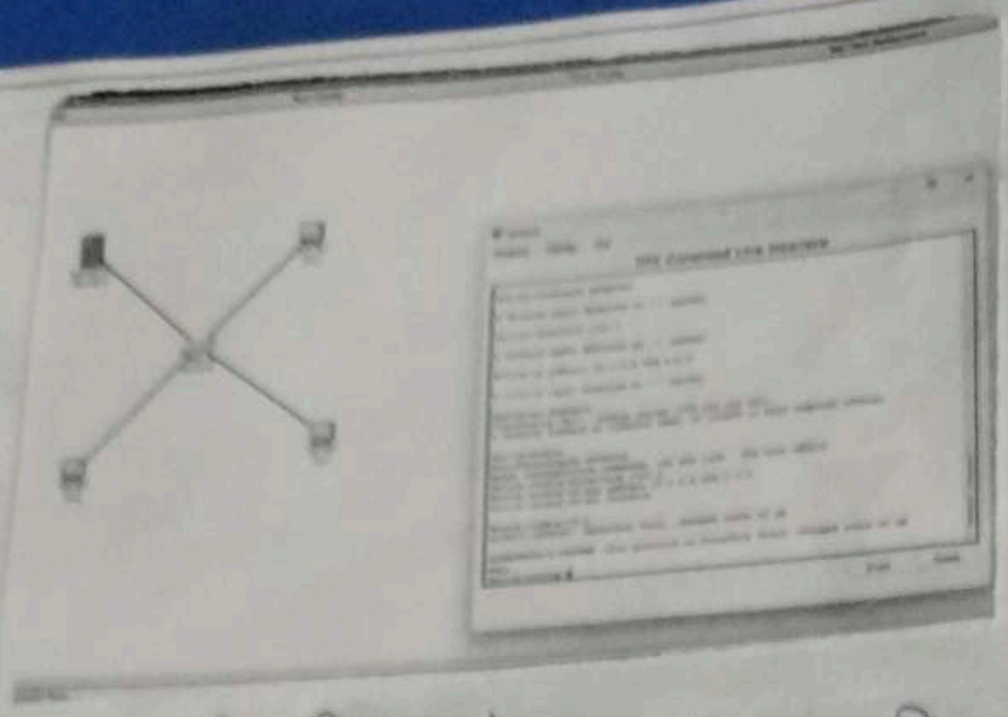
18. Switch to Simulation mode (bottom-right of the workspace)

19. Send a ping or web request between 2 devices & use Play/Capture controls to step through packet as it travels hop-by-hop through switch, observing MAC-address-based forwarding.

7. Observation:

All 4 PCs were successfully connected to switch in star topology & assigned unique static IP addresses within the same subnet (10.0.0.0/25.0.0.0). Ping tests b/w all devices returned successful replies, confirming end to end connectivity.

11/8/26



Steps: Verify Connectivity Using Ping

11. Click PC → Desktop tab → Command Prompt
12. Use ping command to test connectivity to other PCs, eg. ping 10.1.1.2.
13. Successful "Reply from" messages confirm connectivity

Step6: Demonstrate Data Sharing (Server Simulation)

14. Add a Server device (End devices → Server-PT) & connect it to switch.
15. Assign IP an IP address (eg: 10.1.1.10, subnet mask 255.0.0.0) via Desktop → IP Configuration tab
16. Open server's Services (Config) tab & enable HTTP Service
17. From another PC, open Web browser (Desktop tab) & enter server's IP address to load to the page, confirming successful data transfer across the LAN

Step1: Place the Devices

1. Open Cisco Packet Tracer
2. From device category panel, select End Devices & drag 4 PCs onto the workspace.
3. Select switches & drag 1 Switch.

Step2: Cable the Devices.

4. Click Connections → Select Copper Straight-Through cable.
5. Click each PC, then click the switch & select an available Fast Ethernet port to complete each connection.
6. Repeat all 4 PCs, Green link lights confirm a successful physical connection.

Step3: Assign IP Addresses to PCs

7. Click PC → Desktop tab → IP configuration
8. Enter Static IP Address & Subnet Mask as per the IP table (Section 5) for each PC

Step4: Configure Switch Management IP (optional)

9. Click switch → CLI tab
10. Enter commands to assign management IP:

```
enable
configure terminal
interface vlan 1
ip address 10.1.1.5 255.0.0.0
no shutdown
exit
```

```
with open("channel", "r") as f:
    code = list(f.read().strip())
error = check_hamming(code)
if error != 0:
    print("Error detected at position:", error)
    if code[error-1] == '0':
        code[error-1] = '1'
    else:
        code[error-1] = '0'
    print("Error corrected.")
else: print("No error detected.")
data = ""
for i in range(1, len(code)+1):
    if i & (i-1):
        data += code[i-1]
text = binary_to_text(data)
print("Received text:", text)
```

Output:

sender.py

Enter text: HELLO

Binary data: 010010000100101011000111

Hamming code: 010110001000010101001100110010011111

Receiver.py

No error detected

Received text: HELLO

Receiver.py

Error detected at position: 23

Error corrected

Received text: HELLO

5

```
    return ''.join(code)
text = input("Enter text: ")
binary = text_to_binary(text)
code = hamming_code(binary)
with open("channel", "w") as f:
    f.write(code)
print("Binary data:", binary)
print("Hamming code:", code)
print("Data saved in channel file.")
```

6.

2. Receiver Program:-

```
def check_hamming(code):
    n = len(code)
    r = 0
    while 2**r < n+1:
        r += 1
    error = 0
    for i in range(r):
        p = 2**i
        count = 0
        for j in range(1, n+1):
            if j & p and code[j-1] == '1':
                count += 1
        if count % 2 != 0:
            error += p
    return error
```

28/9

Exp-6

i) Aim: Write a program to implement error detection and correction using HAMMINGcode concept, Make a test run to input datastream and verify error correction failure.

ii) Sender Program:

```
def text_to_binary(text):
    return ''.join(format(ord(ch), '08b') for ch in text)

def hamming_code(data):
    m = len(data)
    r = 0
    while 2**r < m+r+1:
        r += 1
    total = m+r
    code = ['0'] * total
    j = 0
    for i in range(1, total+1):
        if i & (i-1):
            code[i-1] = data[j]
            j += 1
    for i in range(r):
        p = 2**i
        count = 0
        for j in range(1, total+1):
            if j & p and code[j-1] == '1':
                count += 1
        code[p-1] = str(count % 2)
```

5 Create a Filter to display only IP/ICMP packets and inspect the packets. Procedure:

* Select Local Area Connection in Wireshark
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture
* Search ICMP/IP packets in search bar.
* Save the packets.

6. Create a Filter to display only DHCP packets and inspect the packets. Procedure:

* Select Local Area Connection in Wireshark
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click start capture
* Search DHCP packets in search bar
* Save the packets.

28/9

i) Aim: Wri
and cor
Make a
error

ii) Sender

* Save the packets.

2) Create a Filter to display only ARP packets and inspect the packets. Procedure:

* Go to capture option
* Seled stop capture automatically after 100 packets.
* Then click start capture
* Search ARP packets in search bar.
* Save the packets.

3) Create a Filter to display only DNS packets and provide the flow graph. Procedure.

* Go to capture option.
* Select stop capture automatically after 100 packets.
* Then click Start capture
* Search DNS packets in search bar.
* To see flow graph click statistics Flow graph
* Save the packets

4) Create a filter to display only HTTP packets and inspect the packets Procedure

* Select Local Area Connection in Wireshark
* Go to capture option
* Select stop capture automatically after 100 packets.
* Then click Start capture.
* Search HTTP packets in search bar
* Save the packets

EXP-8

i) Aim: To Discover Live Hosts Using Nmap Scans (ARP, ICMP, TCP/UDP) on TryHackMe Platform
Room Link: https://tryhackme.com/room/nmap01

ii) Introduction:-

Nmap (Network Mapper) is a free, open-source tool used to discover live hosts, scan ports and identify network services. Host discovery helps identify active systems before port scanning.

Types of Host Discovery

1. ARP Scan: Uses ARP requests to discover live hosts on the same subnet
2. ICMP Scan: Uses ping requests to identify active hosts.
3. TCP SYN Ping: Sends TCP SYN packets and checks for responses.
4. TCP Ack Ping: Sends TCP Ack packets to detect live hosts.
5. UDP Ping: Uses UDP Packets to identify active hosts.

```
if not error:
    with open("Receiver Buffer.txt", "w") as file:
        file.write(f"Ack: {expected_frame}")
    print(f"Ack: {expected_frame}")
time.sleep(2)
```

Output:

Received Frames:

Frameno: 1, DATA: H1

Frameno: 2, DATA: e

Frameno: 3, DATA: l
Frame3 accepted
Ack: 4 sent

Frames Received:

FrameNo: 3 DATA: L
Frame 3 accepted

FrameNo: 4 DATA: O
Frame 4 accepted
Ack: 5 sent

Frames Sent:

FrameNo: 0 DATA: H
FrameNo: 1 DATA: E
FrameNo: 2 DATA: L

Receives Response: Ack: 3
Acknowledgement Number: 3
Ack is correct

receiver.py

```
import Ame
expected_frame=1
while True:
  try:
    with open("Sender_Buffer.txt", "r") as file:
      lines= file.readlines()
  except FileNot.FoundError:
    print("Sender_Buffer.txt not found!")
  time.sleep(2)
  continue
if not lines:
  time.sleep(2)
  continue
print("\n Received Frames:")
error=False
  if frame_no!=expected_frame:
    print(f"Error! Expected Frame{expected_frame}"
          f" but received Frame {frame_no}")
  with
  open("Receiver_Buffer.txt", "w") as
file:
  file.write(f"NACK: {expected frame}")
    error= True
    break
expected_frame+=1
```

Outp

```
try:
    with open("Receiver_Buffer.txt", "r") as file:
        response = file.read().strip()
except FileNotFoundError:
    print("Receiver_Buffer.txt not found!")
    continue

print("Receiver Response:", response)
if response.startswith("ACK"):
    ack_no = int(response.split(":")[1].strip())
    if ack_no == start + len(window)+1:
        print("ACK received correctly.")
        start += len(window)
    else:
        print("Unexpected ACK. Resending window.")
elif response.startswith("NACK"):
    nack_no = int(response.split(":")[1].strip())
    print("Resending from that frame...")
    start = nack_no - 1
else:
    print("Invalid response. Resending...")
print("\n All frames transmitted successfully!")
```

EXP-7

i) Aim:- Write a program to implement flow control at datalink layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another

ii) sender.py

```
import time
window_size= int(input("Enter Window Size:"))
message= input("Enter Text to Message:")

frames=[]
for i, ch in enumerate(message, start=1):
    frames.append((i, ch))
start=0
while start < len(frames):
    window= frames[start:start + window_size]
    with open("Sender_Buffer.txt", "w") as file:
        for frame_no, data in window:
            file.write(f"Frame No:{frame_no}, DATA:{data}\n")
print("\n Frames Sent:")
for frame_no, data in window:
    print(f"Frame No:{frame_no}, DATA:{data}")
print("\n Waiting for Ack...")
```

PRACTICAL 9: IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER

i) Aim:

To implement classless IP subnetting using Cisco Packet Tracer and test network connectivity between devices

ii) Theory:

* Subnetting is the process of dividing a large network into smaller subnetworks. It helps in efficient utilisation of IP address and improves network management.

* The subnet mask for /27 is 255.255.255.224. It provides 8 subnets, with 32 addresses per subnet and 30 usable host addresses per subnet.

iii) Procedure:

i) Step 1: Creating Network Topology

- Open Cisco Packet Tracer and create a new network topoly.
- Add the required routers, switches and PCs from the device selection panel.
- Arrange devices in the workspace and connect them using suitable cables.

* Enter the WEP key 0123456789 click Connect
* Repeat the same process for PC1 and PC2

iii) Expected Output

* All three PCs connect to Mother Network wireless network
* WEP security is configured on the router
* Each PC has its assigned static IP addres

iv) Result:

Thus, A wireless LAN was configured successfully using Cisco Packet Tracer by connecting three PCs to a secured Linksys wireless router

Practical-8: Configuration of Wireless LAN using Cisco Packet Tracer

i) Aim:- To configure a Wireless LAN using Cisco Packet Tracer by connecting three PCs to a Linksys wireless router.

ii) Procedure:

* Place one Linksys wireless router & three PCs (PC0, PC1, PC2) in Cisco Packet Tracer.
* Open router & configure the username & password as admin under Administration tab.
* Under Wireless tab, set the SSID to MothesNetwork
* Configure static IP addresses on all three PCs under Desktop → IP Configuration → Static

IP Configuration:

Device	IP Address	SubnetMask	Default Gateway
Router	192.168.0.1	255.255.255.0	-
PC0	192.168.0.2	255.255.255.0	192.168.0.1
PC1	192.168.0.3	255.255.255.0	192.168.0.1
PC2	192.168.0.4	255.255.255.0	192.168.0.1

* On each PC, open Desktop → PC Wireless → Connect
* Refresh available networks & select Mothersnetwork

1. ARP (Address Resolution Protocol)

i) How many devices can see ARP request?

Ans: 4

ii) Did computer6 receive the first ARP request?

Ans: No

iii) Did computer6 reply to the second ARP request?

Ans: Yes

2. Enumerating Targets

i) What is first IP address Nmap would scan if target is 10.10.12.13/29?

Ans: 10.10.12.8

3. ICMP (Internet Control Message Protocol)

i) What packet is received before sending ping?

Ans: ARP Response.

ii) What is the first device the responded to first ARP request?

Ans: Routes

4. b Reverse-DNS Lookup

). What option should be added to perform reverse-DNS Lookups for all possible hosts

Ans: -R

Target Specification

Targets can be specified as:

* List: Specific IP addresses or hostnames
* Range: A range of IP addresses
* Subnet: An IP network range

Useful Commands:

- nmap -sn TARGETS - Discovers live hosts without port scanning.
- nmap -sL TARGETS - Lists targets without scanning
- nmap -n TARGETS - Disables DNS resolution

Important Nmap Options

Option	Purpose
-PR	ARP scan
-PE	ICMP Echo scan
-PP	ICMP Timestamp scan
-PM	ICMP Address Mask scan
-PS80	TCP SYN ping on port 23
-PA	TCP Ack ping
-R	Reverse-DNS lookup for all targets

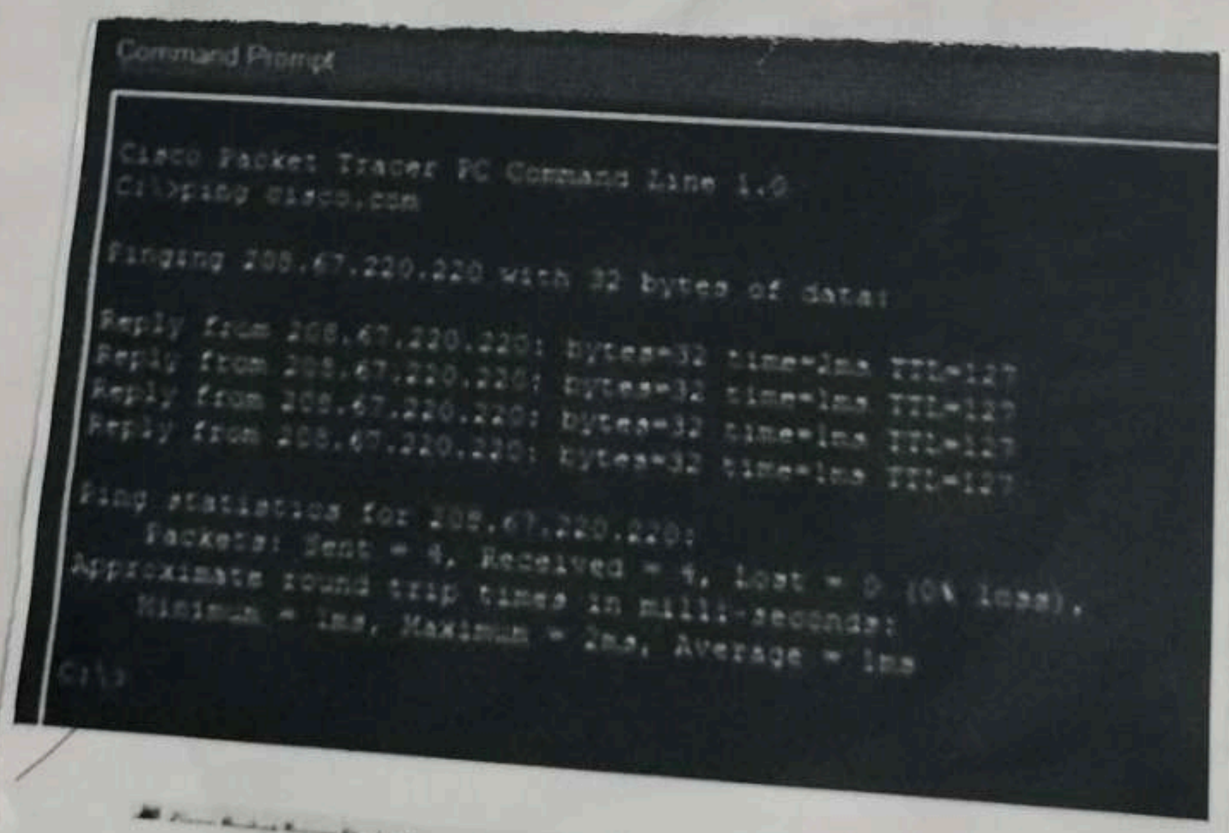
EXP10B: Internetworking Using Wireless Routers, DHCP Servers & Internet Cloud.

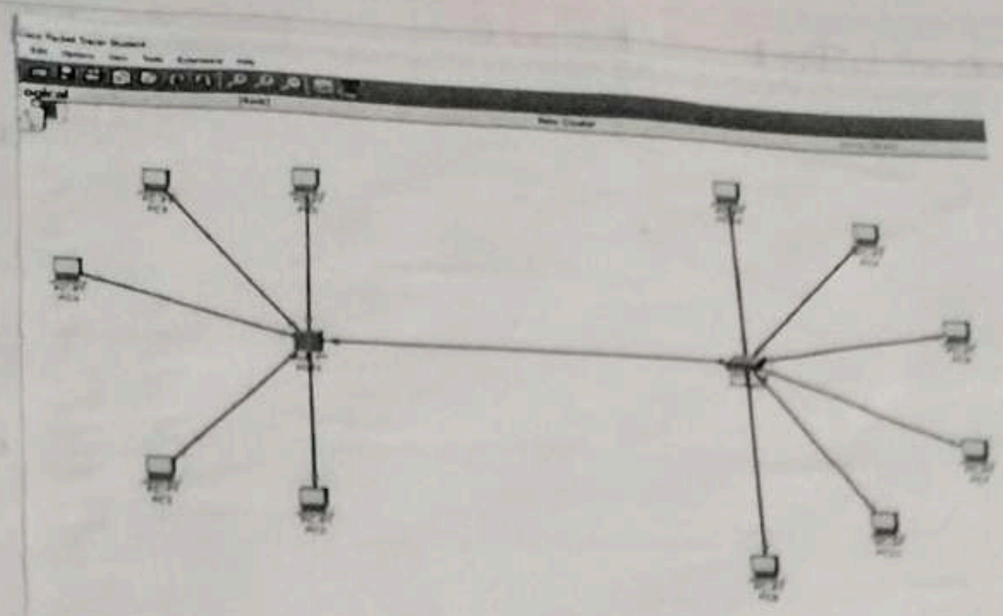
i) Aim: To design & configure an internetwork using wireless routers, DHCP servers in Cisco.

ii) Procedure:-

* Open Cisco and add PC, wireless router, laptop, cable modem with suitable cables.

* Open wireless router GUI & set the SSID to HomeNetwork. Enable the DHCP server & configure DNS server address as 208.67.220.220. Save the settings.





PC0 — Command Prompt (ping)

```
C:\> ping 20.0.0.2

Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=1ms TTL=126
Reply from 20.0.0.2: bytes=32 time=1ms TTL=126
Reply from 20.0.0.2: bytes=32 time=1ms TTL=126
Reply from 20.0.0.2: bytes=32 time=1ms TTL=126

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 2ms, Average = 1ms

C:\> tracert 20.0.0.2
Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1      1 ms      1 ms      1 ms    10.0.0.1
  2      2 ms      2 ms      2 ms    192.168.1.253
  3      3 ms      3 ms      3 ms    20.0.0.2

Trace complete.
```

Result:-

Thus a simple internetwork was designed and configured using router in Cisco Packet Tracer and connectivity was tested using PDU

Steps: Configuring the PCs

Configure the PCs with following static IP addresses

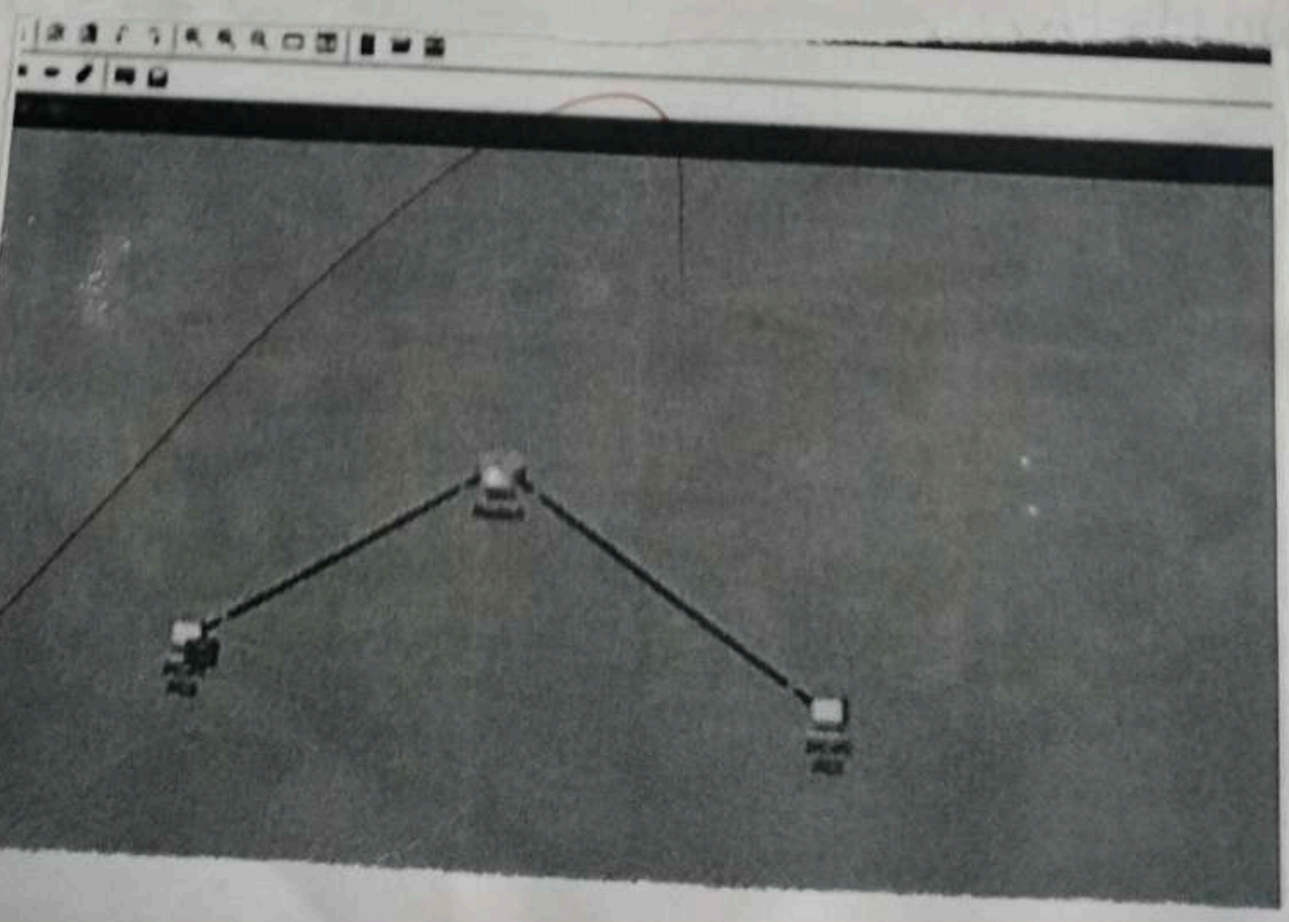
Device	IP Address	SubnetMask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Step3: Connecting the Devices.

Connect PC0 to Router1's FastEthernet0/0 port and PC1 to FastEthernet0/1 using copper straight through cables.

Step4: Testing Connectivity:-

Use Simple PDU tool to send a packet from PC0 to PC1 & check whether it is delivered successfully.



10: Internetworking and Routers in Cisco Packet Tracer.

i) Aim:

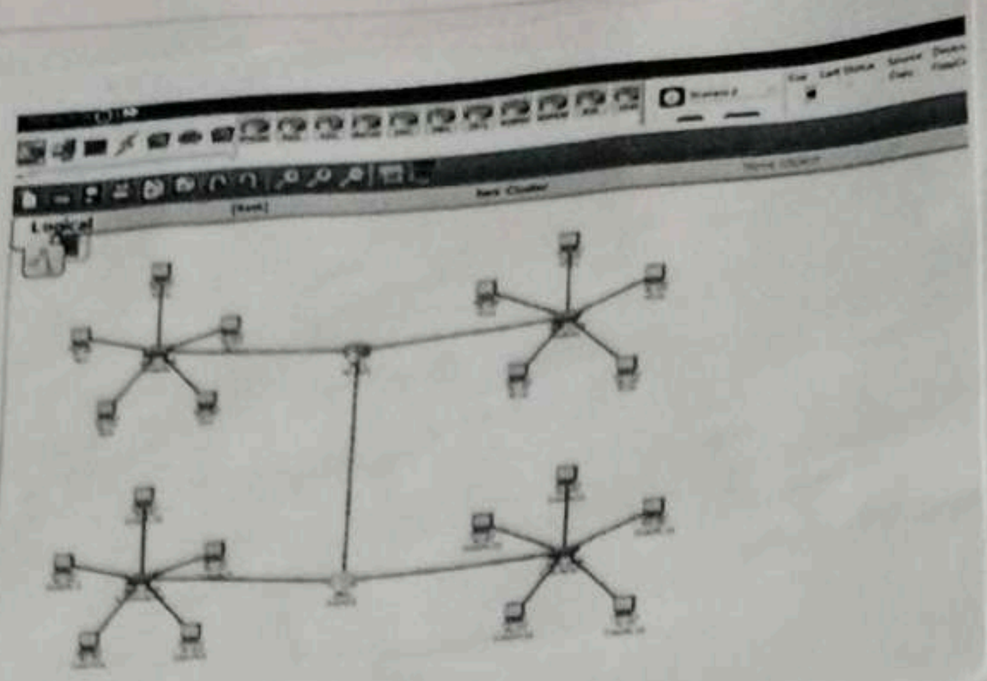
To design and configure a simple internetwork using routers in Cisco Packet Tracer.

ii) Procedure:

Step 1: Configuring the Router

Place on router and two PCs in Cisco Packet Tracer. Open router CLI & configure its interfaces with IP addresses.

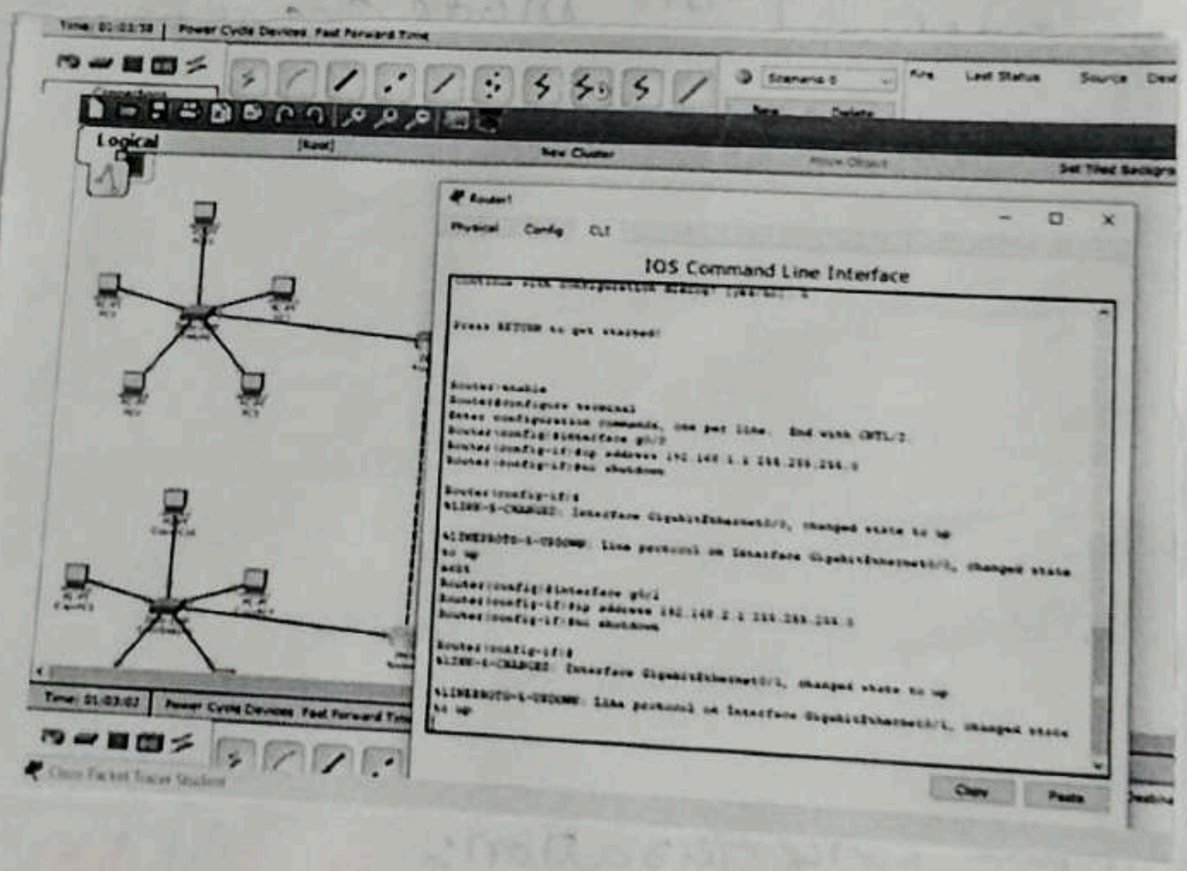
```
enable
configure terminal
interface FastEthernet 0/0
ip address 192.168.10.1 255.255.255.0
no shutdown
exit
interface FastEthernet 0/1
ip address 192.168.20.1 255.255.255.0
no shutdown
exit
```



vi) Step6: Testing Network Connectivity:

After configuring all devices, open command prompt on PC1 & execute following command to test connectivity with PC2

ping 192.168.1.12

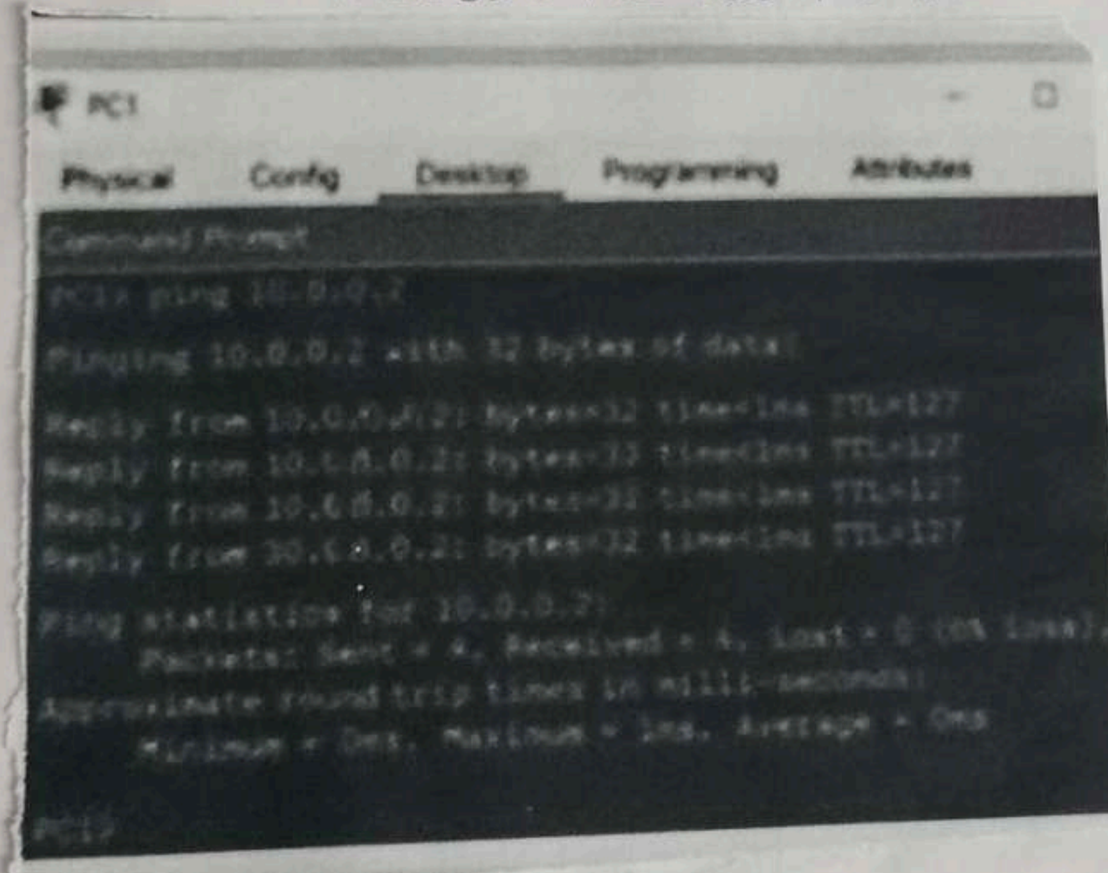


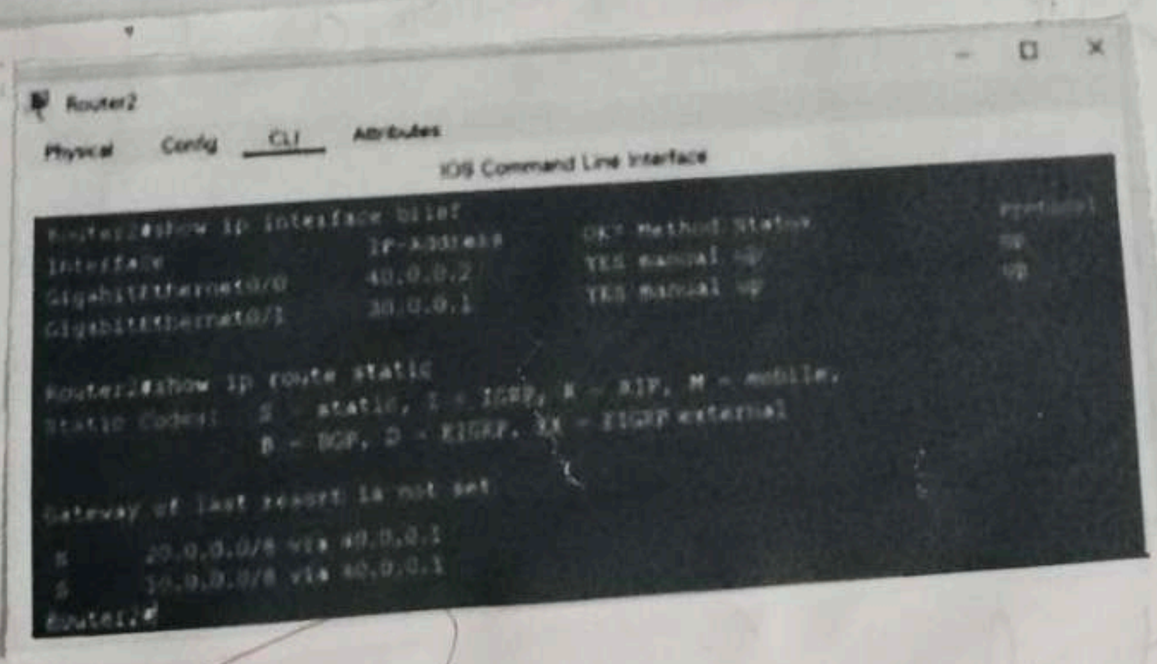
vii) Result:

Thus, classless IP subnetting was studied and implemented using Cisco Packet Tracer. The devices were configured with suitable IP addresses and subnet masks.

Step3: Configuring RIP on Routes()
Enter following commands in Routes() CLI:
enable
configure terminal
routes rip
network 10.0.0.0
network 192.168.1.252

Step4: Configuring RIP on Router1
enable
configureterminal
router rip
network 192.168.1.244
network 192.168.1.248





iii) Result:- Thus, RIP routing was configured and simulated using Cisco Packet Tracer. Network Connectivity & automatic route selection were tested.

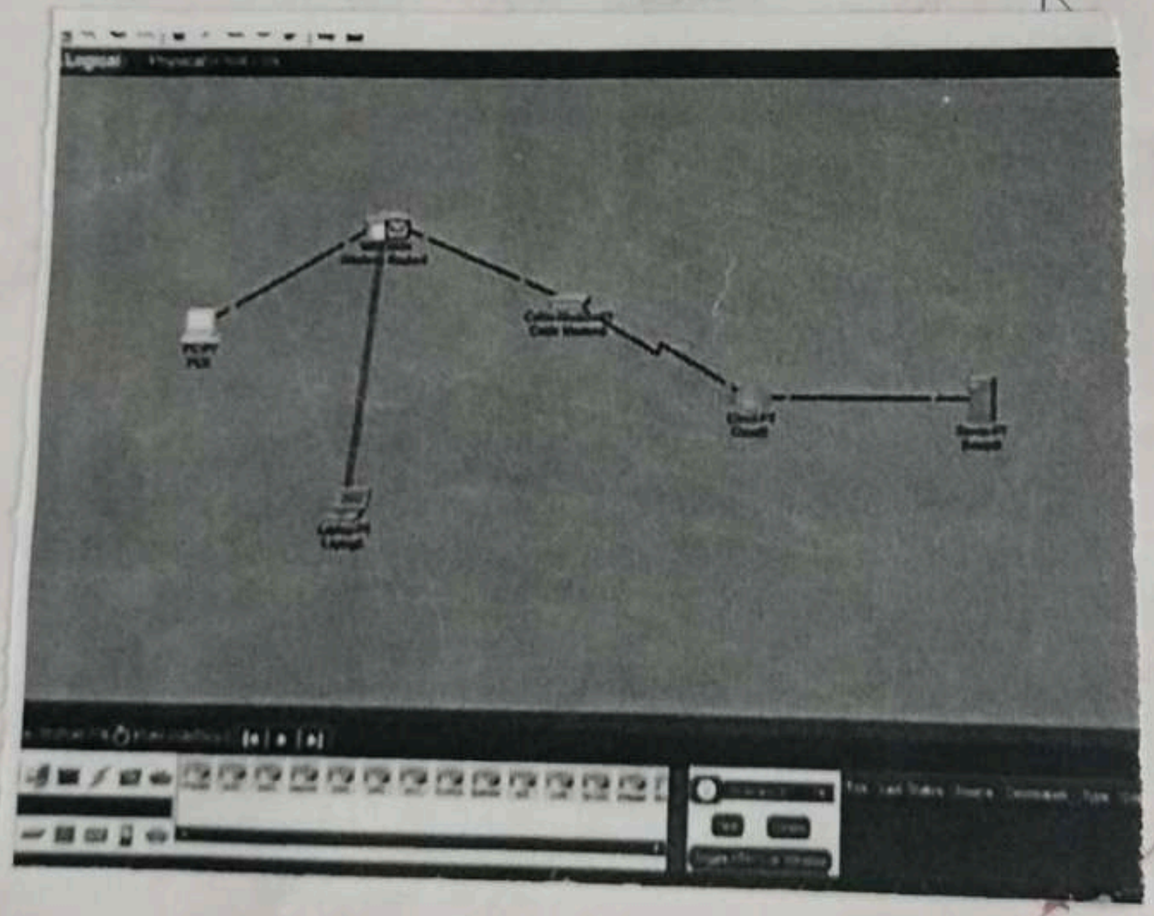
Exp 11: Simulation of RIP Using Cisco Packet Tracer

i) Aim: To configure & simulate the Routing Information Protocol (RIP) using CISCO Packet Tracer

ii) Procedure:

Step 1: Creating the Network Topology
Create a topology with 3 routers (Router0, Router1, Router2) and two PCs (PC0, PC1)
Connect the devices using Fast Ethernet and serial cables.

Step 2: Configuring IP Addresses
Assign the following IP addresses to the PCs and router interfaces.



```
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     40.0.0.1        YES manual up                    up
GigabitEthernet0/1     10.0.0.1        YES manual up                    up
GigabitEthernet0/2     20.0.0.1        YES manual up                    up

Router#show ip route static
Codes: S - static, I - IGRP, R - RIP, M - mobile,
       B - BGP, D - EIGRP, EX - EIGRP external

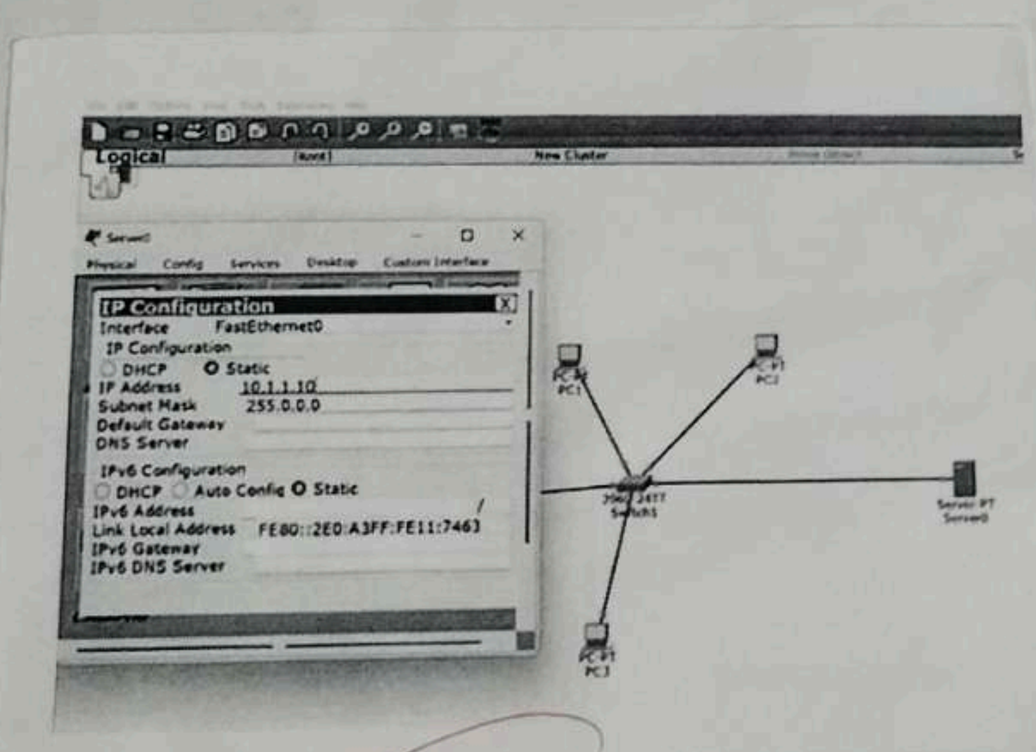
Gateway of last resort is not set

S       40.0.0.2/8 via 20.0.0.2
S       10.0.0.0/8 via 10.0.0.2
Router#
```

Step 4

Step 3: Configuring static Routes on Router 2
Enter the following commands:
enable
configure terminal
ip route 10.0.0.0 255.0.0.0 40.0.0.1
ip route 30.0.0.0 255.0.0.0 50.0.0.2
end
show ip route static

Step 4: Verifying Static Routing
* Use the show ip route static command to verify the configured routes.
* Use the ping or tracert command to test connectivity between networks.
* Disconnect the primary link to check whether the backup route works.



Result:
Thus, static routing was configured and simulated using Cisco Packet Tracer & network connectivity was verified.

Exp11: Simulation of Using Cisco Packet Tracer

i) Aim: To configure and simulate static routing using Cisco Packet Tracer.

ii) Theory:

Static routing is a method of manually configuring routes in table. Backup routes can also be configured using Administrative Distance (AD) values.

iii) PROCEDURE:

Step1: Creating Network Topology.

Create a network topology using three routers and connect them accordingly to given network diagram. Configure the IP addresses of the routes interfaces

Step2: Configuring Static Routes on Router()

Open Router() CLI and enter following commands:

```
enable
configure terminal
ip route 30.0.0.0 255.0.0.0 20.0.0.2 10
ip route 30.0.0.0 255.0.0.0 40.0.0.20
end
show ip route static
```

* Install wireless module if required. Open Desktop → PC Wireless, select Home Network and connect to the wireless network.

* Open Desktop → IP configuration & select DHCP Use the ipconfig/all command to verify that the PC receives an IP address, subnet mask, default gateway.

* Configure the server with static IP address 208.67.220.220 & subnet mask 255.255.255.0. Set DNS record name to Cisco.com & its address to 208.67.220.220

* Open the PC command prompt & execute:

```
ipconfig/release
ipconfig/renew
ping Cisco.com
```

Successful ping replies indicate connectivity to the Cisco.com server

(iii) Result:

Thus, an internetwork using a wireless router, DHCP server, and Internet cloud was configured in Cisco Packet Tracer and connectivity was tested successfully.

EXP14: Write code using RAW sockets to implement packet sniffing

```
from scapy.all import sniff
from scapy.layerss.inet import IP, TCP, UDP, ICMP

def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst

protocol_name = ""
    if protocol == 1:
        protocol_name = "ICMP"
    elif protocol == 6:
        protocol_name = "TCP"
    elif protocol == 17:
        protocol_name = "UDP"
else:
    protocol_name = "UnknownProtocol"
    print(f"Protocol: {protocol_name}")
    print(f"SourceIP: {src_ip}")
    print(f"Destination IP: {dst_ip}")
print("-" *50)
```

sniff(if
filt

Output

```
C:\Users\tkala\D
Protocol: ICP
Source IP: 57.15
Destination IP:

Protocol: ICP
Source IP: 192.
Destination IP:

Protocol: ICP
Source IP: 192
Destination IP

Protocol: TCP
Source IP: 192
Destination I

Protocol: ICP
Source IP: 57
Destination
```

13: Implement your own ping program.

```
import os
import platform
import subprocess

def ping(host="google.com", count=4):
    param="-n" if platform.system().lower()=="windows"
    else
    "-c"

try:
    output= subprocess.check_output(command,
            universal_newlines=True)
    print(output)
except Exception as e:
    print(f"Ping failed:{e}")

if __name__=="__main__":
    ping("8.8.8.8").
```

Output:

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=6ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Reply from 8.8.8.8: bytes=32 time=5ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 5ms, Maximum = 9ms, Average = 7ms
```

UDP

```
import socket
import threading
def udp_server():
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
while True:
    msg = data.decode()
    if msg.lower() == 'quit':
print(f"[SERVER] Client {addr} ended
the connection.")
    break
    s.sendto(data, addr)
s.close()
print("[SERVER] Stopped.")
if __name__ == "__main__":
    threading.Thread(target=udp_server,
        daemon=True).start()udp_client()
```

Output:

```
Enter message (or 'quit' to stop): [SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi
Echo from Server: hi
Enter message (or 'quit' to stop): hello
[SERVER] Received from ('127.0.0.1', 49670): hello
Echo from Server: hello
Enter message (or 'quit' to stop): how are you
[SERVER] Received from ('127.0.0.1', 49670): how are you
Echo from Server: how are you
Enter message (or 'quit' to stop): quit
[CLIENT] Closing connection...[SERVER] Client ('127.0.0.1', 49670) ended the connection.

[CLIENT] Stopped.[SERVER] Stopped.
```

```
while True:
    msg = input("Enter message (or 'quit' to exit): ")
    if msg.lower() == 'quit':
        break
    print(f"[SERVER RESPONSE] {response}")

if __name__ == "__main__":
    server_thread = threading.Thread(target=
        start_server, daemon=True)
    start client
```

Output:

Output:

```
================== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 53458)Enter message (or 'quit' to exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit):
================== RESTART: C:/Users/tkala/Downloads/tcp.py ==========
[SERVER] Listening on 127.0.0.1:5000[CLIENT] Connected to server

[SERVER] Connected by ('127.0.0.1', 60328)Enter message (or 'quit' to exit):
HI
[SERVER] Received: HI
[SERVER RESPONSE] Server received: HI
Enter message (or 'quit' to exit): quit
[SERVER] Connection closed by client.
```

EXP12: End-End Communication at Transport layer

a) Implement echo client-server using TCP/UDP sockets
b) Implement chat program using socket programming

TCP:

```
import socket
import threading
def start_server()
    server= socket.socket(socket.AF_INET, socket.
                          SOCK_STREAM)
server.listen(1)
print("[SERVER] Listening on 127.0.0.1:5000")
conn, addr= server.accept()
print(f"[SERVER] Connected by {addr}")
while True:
    data = conn.recv(1024).decode()
    if not data or data.lower()=='quit':
        print("[SERVER] Connection closed by client.")
        break
    print(f"[SERVER] Received: {data}")
    conn.send(f"Server received: {data}".encode())
conn.close()
server.close()
def start_client():
    client.connect(('127.0.0.1',5000))
    print("[CLIENT] Connected to server")
```

```
sniff(iface="Wifi", prn=packet_callback,
      filter="ip", store=0)
```

Output:

```
C:\Users\tkala\Downloads>python packet.py
Protocol: TCP
Source IP: 57.155.141.117
Destination IP: 192.168.1.6
----------------------------------------------
Protocol: TCP
Source IP: 192.168.1.6
Destination IP: 57.155.141.117
----------------------------------------------
Protocol: TCP
Source IP: 192.168.1.6
Destination IP: 57.155.141.117
----------------------------------------------
Protocol: TCP
Source IP: 192.168.1.6
Destination IP: 57.155.141.117
----------------------------------------------
Protocol: TCP
Source IP: 57.155.141.117
Destination IP: 192.168.1.6
```

Completed
13/10/26